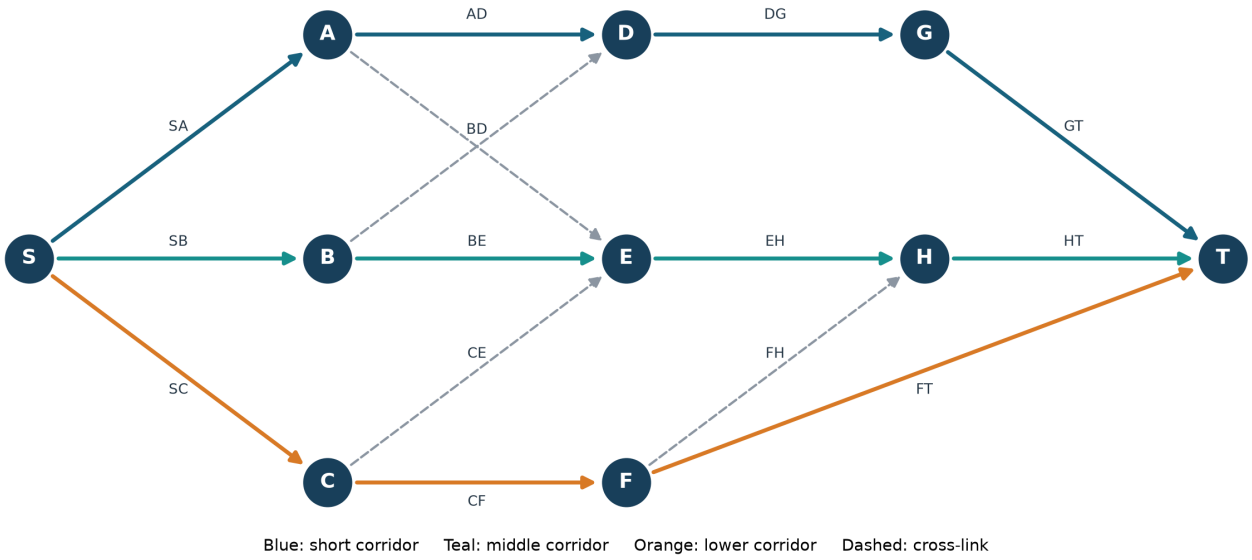
Queue-Aware Shortest-Path Routing

Comparison with Fixed-Weight Dijkstra

Sashwath J | 24BAI0059

Network used in the executable simulation



S is the source and T is the destination. Arrows show the direction of the links.

Ten nodes and 15 directed links provide seven possible S-to-T paths.

What was implemented

Each link has a propagation delay, service capacity and finite FIFO packet buffer.

Fixed Dijkstra uses delay as its link cost. Queue-aware Dijkstra also uses observed queues.

Both methods receive the same workload and random seeds at every offered load.

Example at 2.0 offered packets/tick (mean of 10 matched seeds)

	Fixed	Queue-aware
Average delay (ticks)	9.995	6.833
Packet loss (%)	49.950	0.000
Delivered throughput (packets/tick)	0.999	1.997

At low load (0.5 and 1.0), both methods tie. Under congestion the queue-aware method spreads traffic across routes; frequent route changes are its tradeoff.

Topology parameters and path distances

All values below come from simulate.py in the bottleneck scenario.

Directed link configuration

Link	Direction	Delay	Capacity / FIFO
SA	S -> A	1 tick(s)	1 pkt/tick / 6
AD	A -> D	1 tick(s)	1 pkt/tick / 6
DG	D -> G	1 tick(s)	1 pkt/tick / 6
GT	G -> T	1 tick(s)	1 pkt/tick / 6
SB	S -> B	2 tick(s)	2 pkt/tick / 8
BE	B -> E	1 tick(s)	2 pkt/tick / 8
EH	E -> H	1 tick(s)	2 pkt/tick / 8
HT	H -> T	1 tick(s)	2 pkt/tick / 8
SC	S -> C	2 tick(s)	3 pkt/tick / 10
CF	C -> F	2 tick(s)	3 pkt/tick / 10
FT	F -> T	2 tick(s)	3 pkt/tick / 10
AE	A -> E	3 tick(s)	1 pkt/tick / 6
BD	B -> D	3 tick(s)	1 pkt/tick / 6
CE	C -> E	3 tick(s)	1 pkt/tick / 6
FH	F -> H	3 tick(s)	1 pkt/tick / 6

Possible S-to-T paths

Path	Cost
S-A-D-G-T	4 ticks
S-B-E-H-T	5 ticks
S-A-E-H-T	6 ticks
S-C-F-T	6 ticks
S-B-D-G-T	7 ticks
S-C-E-H-T	7 ticks
S-C-F-H-T	8 ticks

Cost = sum of fixed link delays.

It excludes queue waiting time.

Observed minimum delay is 5 ticks because of per-tick processing order.

Routing rule

Fixed: link weight = configured delay.

Queue-aware: weight = delay x (1 + 2.5 x recent peak queue / FIFO size).

Routes refresh every 5 ticks; packet routes are chosen when packets enter S.

The upper path has cost 4 but capacity 1 packet/tick. Its fixed-weight baseline saturates above that offered rate; other paths have spare capacity.

Experiment and metrics

Nine loads: 0.5 to 3.5 packets/tick; 10 matched seeds; 5,000 ticks/run; 180 runs.

Delay = mean generation-to-delivery time (ticks), for delivered packets.

Loss = dropped/generated; throughput = delivered/ticks (packets/tick).

Packet conservation is asserted: generated = delivered + lost + pending.

References: Dijkstra (1959), DOI 10.1007/BF01386390;

Shen et al. (2016), DOI 10.1016/j.physa.2016.02.002.

Simulation output

```
C:\Users\SASHWATH\Documents\CN Project>python simulate.py --table
Measured means | 10 matched seeds | 5000 ticks/run
Load  Algorithm     Delay(ticks)   Loss(%)   Throughput(pkt/tick)   Route changes
----  ------------  ------------   -------   --------------------   -------------
 1.0  baseline             5.000     0.000                  0.999             0.0
 1.0  queue_aware          5.000     0.000                  0.999             0.0
 1.2  baseline             9.983    16.612                  0.999             0.0
 1.2  queue_aware          5.963     0.002                  1.199           581.1
 2.0  baseline             9.995    49.950                  0.999             0.0
 2.0  queue_aware          6.833     0.000                  1.997           999.0
 3.0  baseline             9.996    66.633                  0.999             0.0
 3.0  queue_aware          7.375    11.133                  2.663           999.0
Completed 80 runs; packet conservation passed in every run.

C:\Users\SASHWATH\Documents\CN Project>
```

What the output shows

At loads 0.5 and 1.0, both methods give the same delay, loss and throughput.

At load 2.0, fixed routing saturates its one-packet/tick upper path:

9.995 ticks delay, 49.950% loss, 0.999 packets/tick delivered.

Queue-aware routing uses the middle and lower paths when the upper queue grows:

6.833 ticks delay, 0.000% loss, 1.997 packets/tick delivered.

Mean assigned packets at load 2.0: upper 3,340; middle 3,330; lower 3,330.

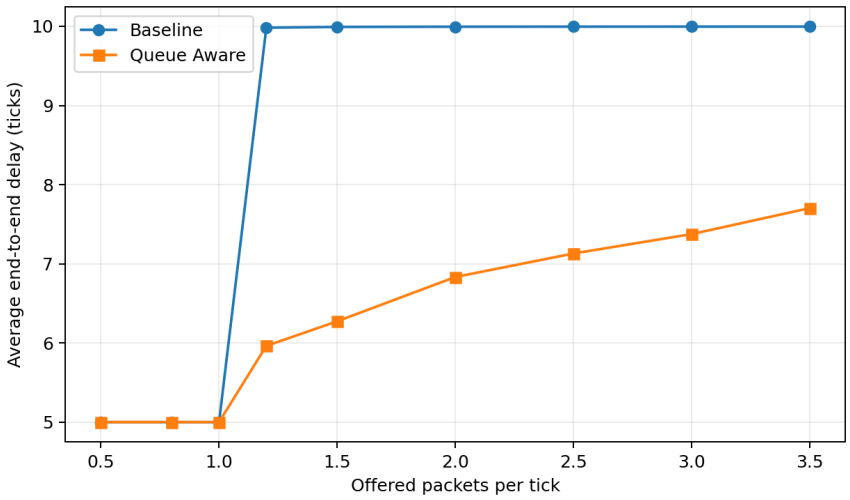
The longer paths cost 5 and 6 ticks, but avoiding long queues lowers
the measured end-to-end delay under this congested load.

At load 3.5, queue-aware loss rises to 17.224%; mean route changes are 999.

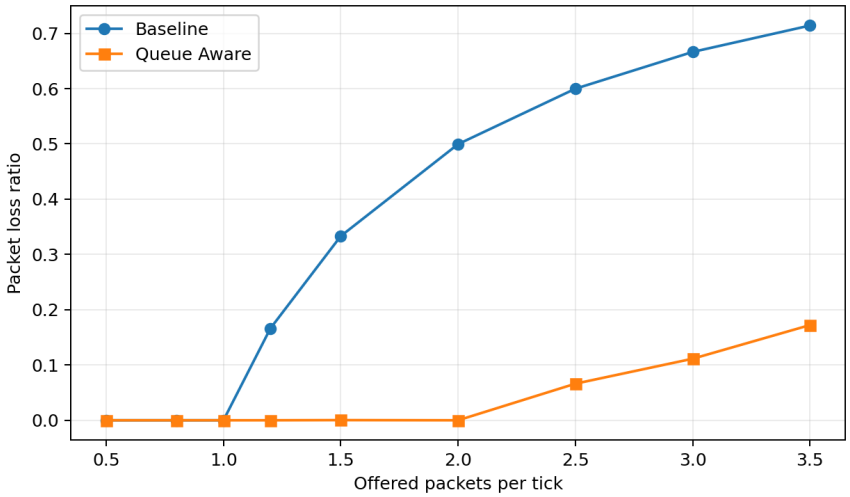
Measured output graphs

Plots are generated by the same source code from the new 180-run experiment.

Average end-to-end delay (ticks)



Packet loss ratio



Delivered throughput (packets/tick)

